Aalto-yliopisto
Aalto-universitetet
Aalto University

Master's Programme in Electronics and Nanotechnology

Development of a unified automated testing suite for human-machine interfaces of uninterruptable power supply systems

Ensuring reusability, reliability, and consistent performance of human-machine interfaces through advanced testing methodologies

Chanchal Kumari

Master's thesis
2025

Copyright ©2025 Chanchal Kumari

Aalto-yliopisto
Aalto-universitetet
Aalto University

Author Chanchal Kumari

Title of thesis Development of a unified automated testing suite for human-machine interfaces of uninterruptable power supply systems

Programme Master's in Electronics and Nanotechnology

Major Photonics and Nanotechnology

Thesis supervisor Prof. Marko Hinkkanen

Thesis advisor(s) Toni Kuikka, MSc

Collaborative partner Eaton Oy

Date 16.09.2025 **Number of pages** 53 **Language** English

Abstract

Human-machine interfaces (HMIs) are one of the critical parts of any machine or system that allows users to communicate with, and control the machinery through graphical displays, buttons, touch screens and multiple other input methods. This thesis presents the development of a unified and reusable solution for automated testing of HMI in Eaton uninterruptable power supply (UPS) products. Motivated by the need for scalable and maintainable testing across diverse HMI platforms, the work addresses three core objectives: developing a flexible tests suite, evaluating suitable testing frameworks and methodologies, and identifying challenges in test reusability and scalability. The proposed solution was validated on two distinct user interface (UI) setups: an application-based UI and a web-based UI. Keyword driven testing was selected due to its simplicity, and the test suite was designed using methods such as error seeding, input partitioning, equivalence class partitioning. To enhance reusability, high-level abstracted test cases were developed. The Robot Framework was employed for the functional testing due to its modularity and ease of use. Additionally, a literature review was conducted to explore emerging trends in artificial intelligence (AI)/machine learning (ML)-driven testing strategies, highlighting both opportunities and challenges, along with proposed solutions.

Keywords Driver testing, functional testing, high-level abstraction, human-machine interface, real-time simulation, test automation, uninterruptable power supply.

Table of contents

Preface and acknowledgements 5
Symbols and abbreviations 6
Symbols 6
Abbreviations 6
1 Introduction 7
1.1 Objectives 8
1.2 Structure of the thesis 9
2 Literature review 10
2.1 Brief introduction of the UPS 10
2.2 Human machine interface (HMI) 13
2.3 Manual testing vs automation testing 16
2.4 Need and challenges of automation testing 17
2.5 Test automation frameworks 19
2.6 Various techniques of testing GUI 21
2.7 Emerging trends: AI-driven and ML-enhanced testing strategies 23
3 Prerequisites for HMI testing 25
3.1 Description for test setup 25
3.2 Test automation scope 27
3.3 Limitations 28
3.4 Reusability of test cases 29
4 Testing frameworks and methods 32
4.1 Tools and frameworks 32
4.2 Testing Methods 34
4.3 Workflow and test architecture 36
5 Results 39
5.1 Test case overview 39
5.2 Result analysis 42
6 Conclusions 48
References 50

Preface and acknowledgements

I would like to express my sincere gratitude to my supervisor, Prof. Marko Hinkkanen, my advisor, Toni Kuikka and my manager Aleksi Karjalainen for their constant support and guidance throughout the course of this thesis. I would also like to extend my gratitude to Eaton for their collaboration on this work, and to my colleagues at Eaton, for their support.

Additionally, a heartful thanks to my family and friends, especially my uncle Palawan Kr. Jha, aunt Kriti Gupta Jha and my close friends including Tushar, Aman, Seshaayini and Rutuja, whose encouragement and understanding meant a great deal to me.

Chanchal Kumari
Martinlaakso, Vantaa
16.09.2025

Symbols and abbreviations

Symbols

V	Voltage

Abbreviations

AC	Alternating Current
AI	Artificial Intelligence
API	Application Programming Interface
AR	Augmented Reality
BDD	Behaviour-Driven Development
CAN	Controller Area Network
CI/CD	Continuous Integration/Continuation Deployment
CPU	Central Processing Unit
CSB	Communication Server Board
DC	Direct Current
FPGA	Field-Programmable Gate Array
GUI	Graphical User Interface
HIL	Hardware-In-Loop
HMI	Human Machine Interface
I/O	Input/Output
LCD	Liquid Crystal Display
MBT	Model-Based Testing
MCU	Machine Control Unit
ML	Machine Learning
PC	Personal Computer
PLC	Programmable Logic Controller
PLD	Programmable Logic Device
POM	Page Object Model
REST	Representational State Transfer
RS-232	Recommended Standard 232
SUT	System Under Test
THD	Total Harmonic Distortion
UI	User Interface
UPM	Universal Power Module
UPS	Uninterruptable Power Supply
USB	Universal Serial Bus
VGT	Visual GUI Testing
VNC	Virtual Network Computing
VR	Virtual Reality

1 Introduction

Human-machine interfaces (HMIs) are one of the critical parts of any machine or system that allows users to communicate with and control the machinery through graphical displays, buttons, touch screens and multiple other input methods. As interaction with complex machines evolve, so does the need for better HMI [1]. HMI presents complex operations happening inside the machinery in easily comprehensible format that the users can monitor and control those operations without having in-depth understanding of the implementations of those operations. HMIs serves multiple purposes such as displaying information, controlling operations, monitoring performance, and data logging. Given its significance, research is required to determine whether the HMI used in the system is working properly and if it is efficient.

In this context, HMIs are used for uninterruptible power supply (UPS) units. A UPS is a device that provides emergency power to a load when the main power source fails. It ensures that critical systems continue to operate without any interruption during power outages, power drops, or other electrical disturbances [2][14][17]. Displays in a UPS system such as liquid crystal display (LCD) panels, present essential information like battery status, load levels, alarm notifications, and real time data visualization in an easy-to-read format. The hardware for these UPSs is complex, including multiple components that each perform complicated tasks. These complexities necessitate thorough testing.

Traditional user interface (UI) testing approaches such as manual testing, platform dependent testing, visual-based testing and script-based automation are common in the testing field. However, they often lack cross-platform testing and are difficult to maintain in dynamic UI environments [4][5]. Given the need for scalable, reliable and cross-platform testing solutions, practices for automated testing of HMIs are growing significantly with the aim to improve quality assurance, user satisfaction, cost efficiency and security [3].

In recent years, many studies have explored cross-platform or universal UI testing solutions. For instance, one survey on automated graphical user interface (GUI) testing strategies highlighted the need of cross-platform test case reusability and the role of artificial intelligence (AI) in test generation to improve the practice of automation testing [6]. These approaches utilize the abstract test logic from UI specification, enabling test reusability across different platforms.

Eaton is the company for which this research was conducted. The main challenge in automating tests for Eaton's UPS HMI system, is the variation in architecture and communication protocols across different UPS product generations, due to improved technology over the years. These differences require separate test setups for each device, making the process inefficient and hard to scale. Furthermore, during product development stages, tests must be rerun frequently to validate ongoing changes and ensure system stability. This highlights the need for a more unified and flexible test automation approach that can adapt to diverse HMI configurations within UPS systems and support continuous testing throughout the development lifecycle.

The three common steps for the development of a product are 1) requirement, 2) design and implementation, and 3) verification and validation [7]. The third stage introduces testing. Testing can be categorize based on the different test levels such as unit testing, integration testing, and system testing [8]. These tests are performed at different levels of system abstraction [9,10,11]. Using test automation methods leads to lesser human errors due to less human interactions [12].

Unit testing involves testing the behaviour of an individual component to ensure that it works as per requirement suggests. On the other hand, integration tests focus on the interaction between integrated units or components and across the layers of the system under test (SUT) implementation [8,9]. Finally, system testing involves testing the whole integrated system to verify that it meets the requirement under various scenarios, also called scenario-based testing [8].

1.1 Objectives

The primary goal of this thesis is to develop unified automated tests for the displays which can be integrated with or expanded to other HMI in different Eaton UPS products, considering all the factors described above. These tests will allow testers to identify bugs and issues in the display and help improve firmware development for the system. To accomplish this task, this thesis utilizes high-level abstraction logics to allow the use of this testing across platforms.

The main research objectives for this thesis can be summarized as follows:

1) Develop a unified and scalable test suite for automated testing of HMI across different systems.
2) Implement and evaluate testing frameworks, methodologies, and tools suitable for testing the HMI in multiple systems.
3) Identify challenges and solutions for the scalability and reusability of automated testing across different HMI.

4) Explore the potential of AI technologies in HMI automated testing for future advancement.

1.2 Structure of the thesis

The rest of this thesis is organized as follows. Chapter 2 reviews the literature covering the various aspects of automation testing and provides a brief description about HMI and UPS. Chapter 3 describes the prerequisites needed for this thesis and test setup that has been used. Chapter 4 introduces the tools, frameworks, and methods implemented in this work based on the need, relevance, and effectiveness and explains the workflow and test procedure followed. Chapter 5 discusses various practical test results and issues encountered. Finally, chapter 6 summarizes the work and results for the objectives outlined in this chapter.

2 Literature review

2.1 Brief introduction of the UPS

A UPS is a device which provides uninterruptable and high-quality power to critical loads in case of power supply failure. The three main responsibilities of an UPS are that it protects hardware damage caused by surges and spikes, prevents data loss and corruption, and power conditioning. UPS provides protection against the common power problems such as power failure, power sag, power surge, undervoltage, overvoltage, electrical noise, frequency variation, switching transient and harmonic distortions [13]. These problems are addressed by utilizing regulated sinusoidal output voltage with low total harmonic distortion (THD), unaffected by variations in input voltage or load. Additionally, when switching between normal power mode and backup power mode (and vice-versa), it should happen instantly without any delay [14].

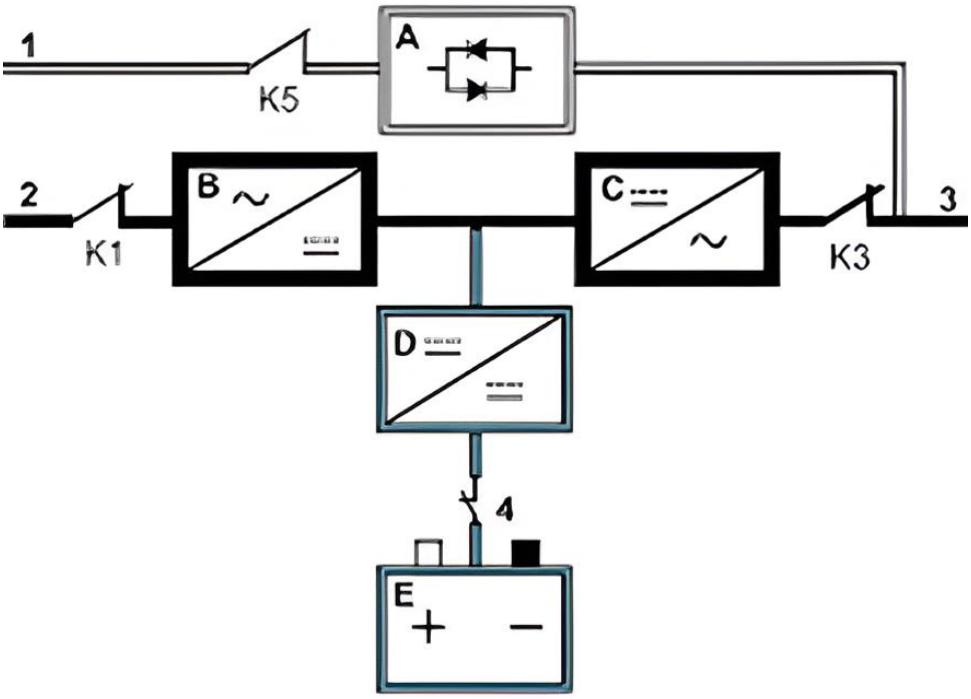


Figure 1: Basic block diagram of UPS. A: Static switch, B: Rectifier, C: Inverter, D: Battery charger, E: Battery, 1: Bypass input, 2: Rectifier input, 3: Output, 4: Battery breaker [16].

UPS systems are connected to an external alternating current (AC) power source such as the utility grid. The main component of the UPS system includes a rectifier (AC/ DC converter), an inverter (DC/AC converter), a battery charger (DC/DC converter), a battery bank, a static switch (bypass) and a load as shown in figure 1. There are multiple topologies such as off-line topology (standby topology), on-line topology, which is also known as double conversion topology, and line-interactive topology [17].

In off-line topology, in the normal mode, the load is directly supplied by the three phase AC source without any power conditioning and battery bank gets charged by the rectifier. It is connected in parallel with the load and stays on standby until the main power supply fails, in which case it activates and supply the load as depicted in figure 2.

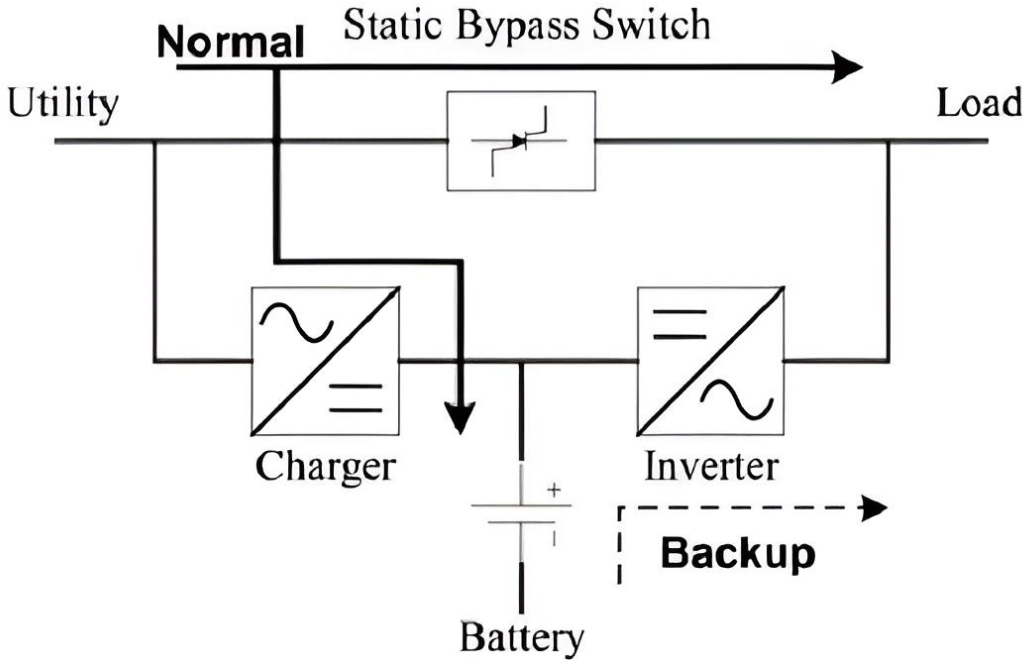


Figure 2: Block diagram of offline UPS [17].

For the online topology, as shown in figure 3, a three-phase AC input is taken from the grid and fed into the rectifier which converts it to DC power. This stable DC output is then fed into the inverter, which converts it back into AC power and supply it to the load. Therefore, the topology is also known as the 'double conversion'.

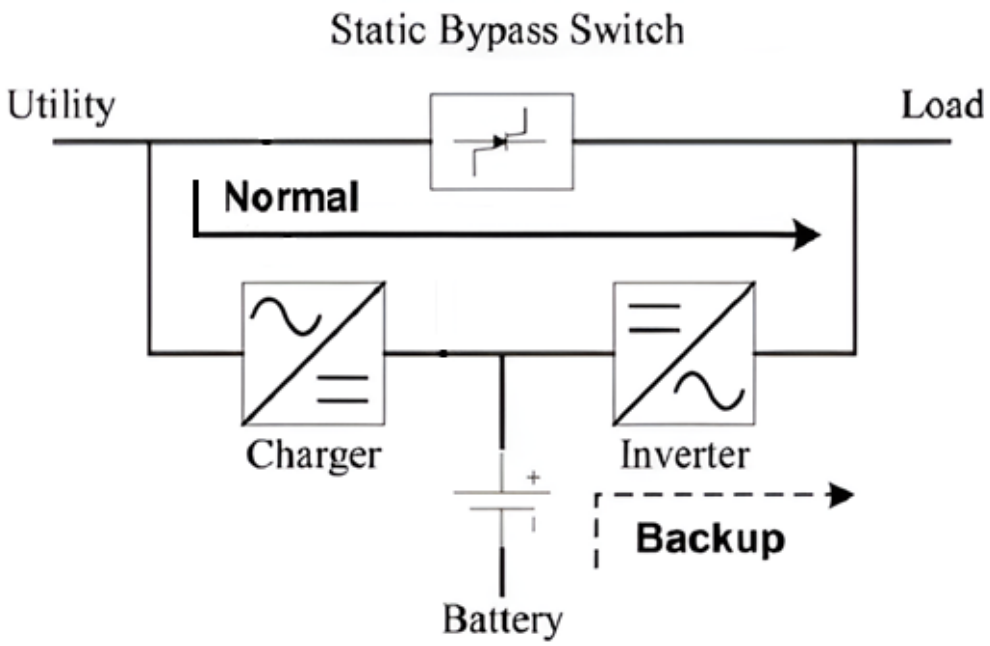


Figure 3: Block diagram of online UPS [18].

In line-interactive topology (figure 4), a static switch allows UPS to seamlessly switch to bypass mode in case of supply unavailability, overload or any faults in the rectifier or inverter components of UPS. A line-interactive UPS can work as either on-line or as an off-line UPS [14].

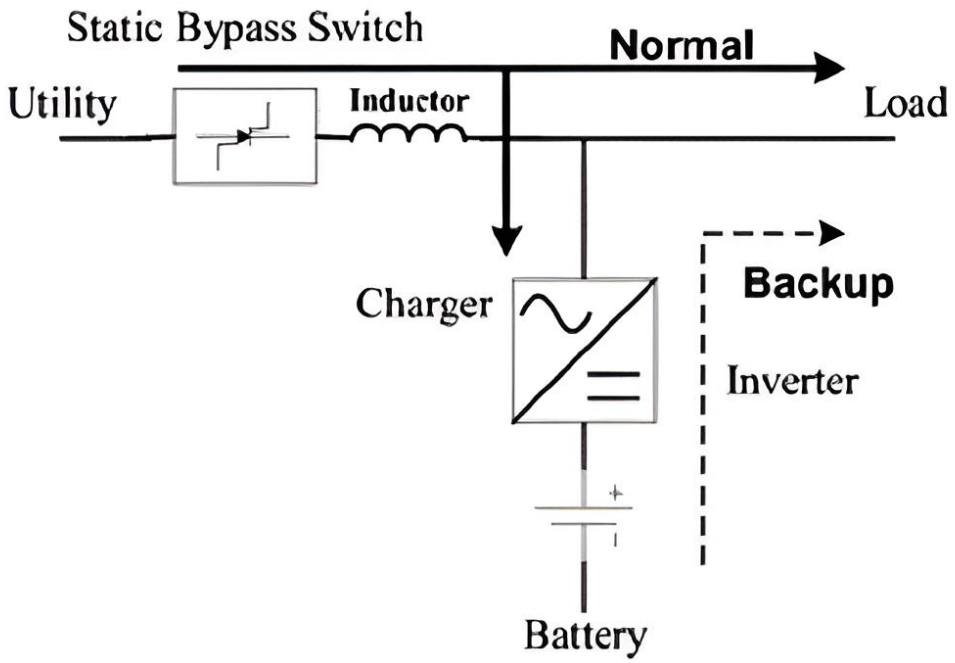


Figure 4: Block diagram of line-interactive UPS [17].

2.2 Human machine interface (HMI)

HMI is an integral part of any machinery. It plays a very critical role in communicating the operations of a machinery to humans. It is needed for the monitoring and controlling purposes of the system. HMI in the UPS system works by collecting the data from components such as programmable logic devices (PLDs), sensors, programmable logic controllers (PLCs), multiple interface boards, and displays it to the user in a graphical format. User can interact and perform operations such as adjusting the settings, initiating tests, or managing power modes of the UPS. HMI also shows real-time data and helps in troubleshooting and maintenance of the UPS by displaying alarms, warnings, and status of the system.

There are many kinds of HMIs nowadays. They can be categorized as touchscreen interface, voice-activated interface, gesture-based interface, virtual reality (VR), augmented reality (AR) interfaces, biometric interfaces, GUIs, and web-based HMI [1][3]. Each one has their own advantages.

The key factors to consider while choosing the best HMI for any system includes [3]:

1) Compatibility with the system it will be integrated into.
2) Hardware of the HMI, based on the environmental factors like temperature, humidity, and vibration.
3) Remote access for the HMI.
4) Scalability for the future system expansion.
5) Screen features such as resolution, brightness, and ability to display complex data.
6) Communication protocol required for the HMI.
7) Screen size, depending on the application's need.
8) Cost effectiveness, supporting budget constraints.

Eaton has different HMI available for various products shown in figure 5, 6, and 7. Mid-range and large UPS products have colour touch displays. However, small UPS's have simple display with few push buttons. Eaton has various kinds of UIs across its range of UPS products. In this thesis, three distinct HMIs from Eaton UPS models, referred to as UPS 1, UPS 2, and UPS 3, are presented to illustrate the diversity in hardware and software architecture.

For instance, UPS 1 has three interface boards, machine control unit (MCU), universal power module (UPM), and communication server board (CSB) which is connected to HMI. MCU supervises all the controls and actions of the UPS. It is used to supervise various controllers, multiple power modules in a modular UPS and ensures smooth interaction between all the modules.

UPM controls rectifier, inverter, and battery operations. It also handles actual power processing while CSB helps to establish communication between the interface boards and HMI. The serial communication interface between the CSB and the display uses the Modbus protocol.

In UPS 2, CSB board is not present. Hence, the HMI is directly connected with the MCU board and communicates via representational state transfer application programming interface (REST API). This setup enables a web-based UI solution, where the UI is accessible through a web browser, giving design flexibility and remote access. In contrast, other UPS products have the HMI application running on the display hardware, where the interface is integrated with the system's embedded software.

The display's images 5, 6, and 7 given below show the differences between the HMI's design elements. The objects, visual elements, colors schemes, and data formats differ with the UPS even though the functional content is same.

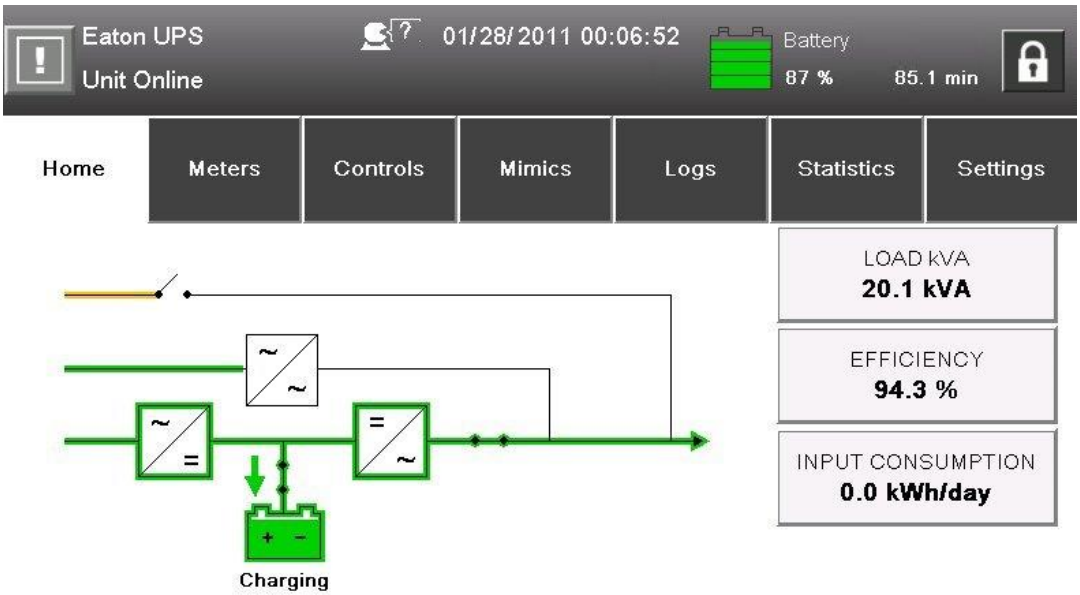


Figure 5: Self-documented Eaton HMI displaying the overview of UPS 1 accessed via remote access.

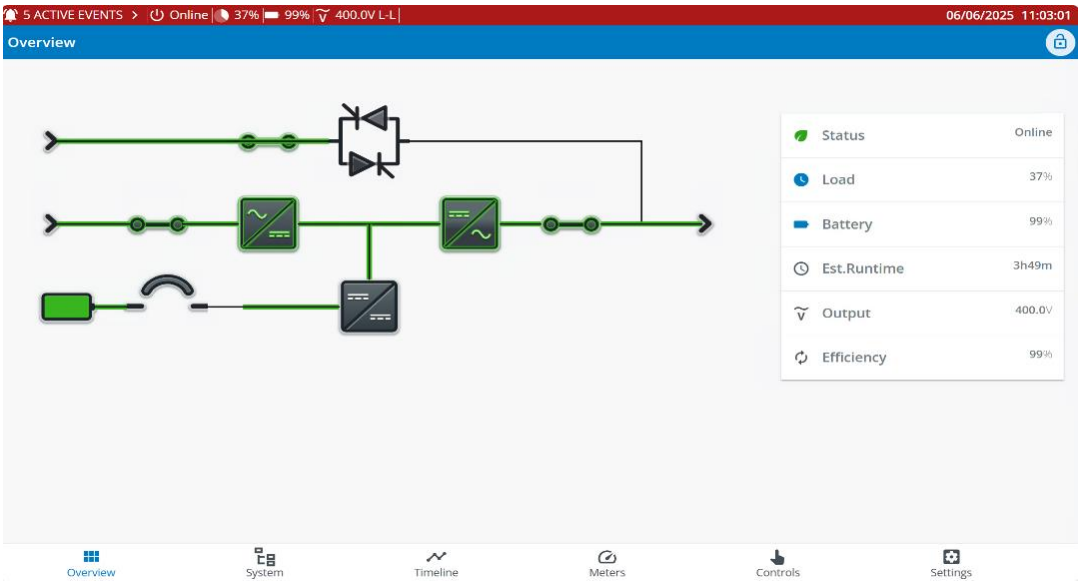


Figure 6: Self-documented Eaton HMI displaying the overview of UPS 2 accessed from hardware-in-loop (HIL).

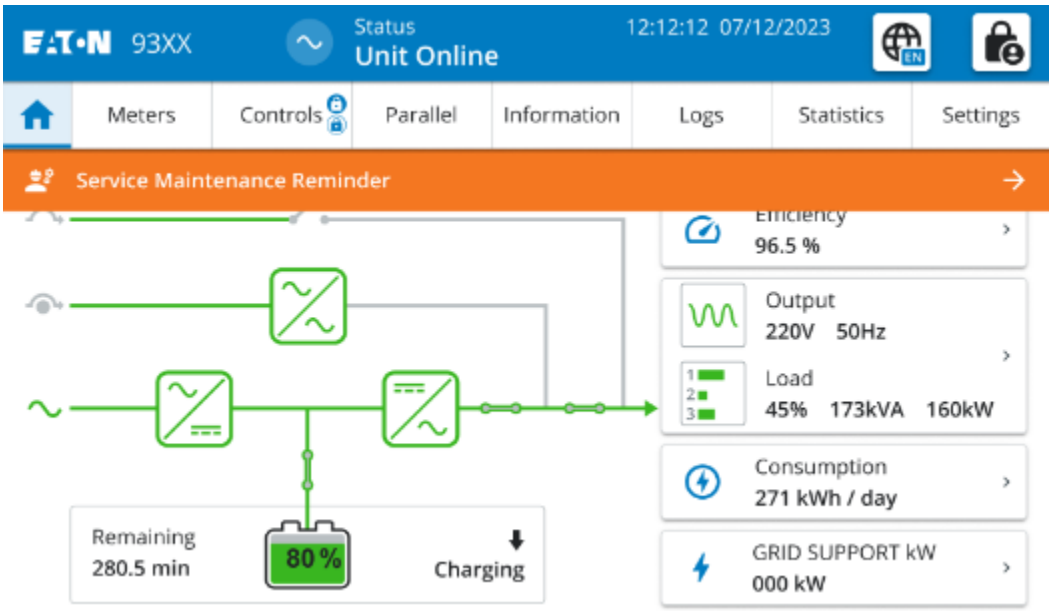


Figure 7: Self-documented Eaton HMI displaying the overview of UPS 3 accessed via remote access.

2.3 Manual testing vs automation testing

Testing the functionality of a system is a crucial step for developing a product free from defects or bugs. Manual testing is a traditional way of testing any software or functionality in a system. Manual testing is basically a type of testing which is performed by a human without any automation tools.

Manual testing is suitable for some scenarios such as [15]:

1) Experimental testing, where test cases are underdeveloped or still evolving, particularly when exploring new features or testing previously fixed bugs.
2) Complex test scenarios, that are too complex to automate or requires human intervention.
3) Rarely executed or high-risk tests.

On the other hand, automation testing is a practice which is done by using automation tools and without any additional human efforts. Automation testing is suitable for repetitive and regression tests that would be time consuming if done manually. Data-driven testing technique involves executing the same tests with different sets of data inputs such as different user credentials, language settings, and ups configuration parameters. It is also useful for stress testing simulating high-frequency interactions or simultaneous inputs, like how multiple users might engage with the HMI in a real-time scenario [15].

To automate the tests effectively, a clear process needs to be defined. For automation testing common sets of steps are defined, which are depicted in the figure 8. These steps include test tool selection, defining scope of automation testing, planning, designing and development, test execution, and test suite maintenance. Step 1 is to select test tools. Nowadays, there are many tools and frameworks available in the market. For example, Selenium, Appium, Playwright, Katalon, Leapwork, and Robot Framework. The tester, referring to the individual responsible for executing and evaluating the tests, can select the suitable tools or frameworks, based on the technology or application that is being tested [19].

In step 2, tester needs to define the scope of the tests, including identifying which tests needs to be included and determining their priority. This decision depends on time constraints, the system under test and the criticality of each test. Step 3 includes planning, designing and development. In this stage, the scope should be defined and system or application under test should be prepared, ensuring it is correctly configured and appropriately connected. Test scripts need to be created, timeline of scripting and execution should be decided. Step 4 is the execution step, where the test scripts are run with

required inputs and the tester gets the detailed test reports afterwards. These reports will be used to identify the bugs and fixing them. Lastly, in step 5 all the tests are organized properly for future maintenances like addition, removal of test cases and modifying the results [19].

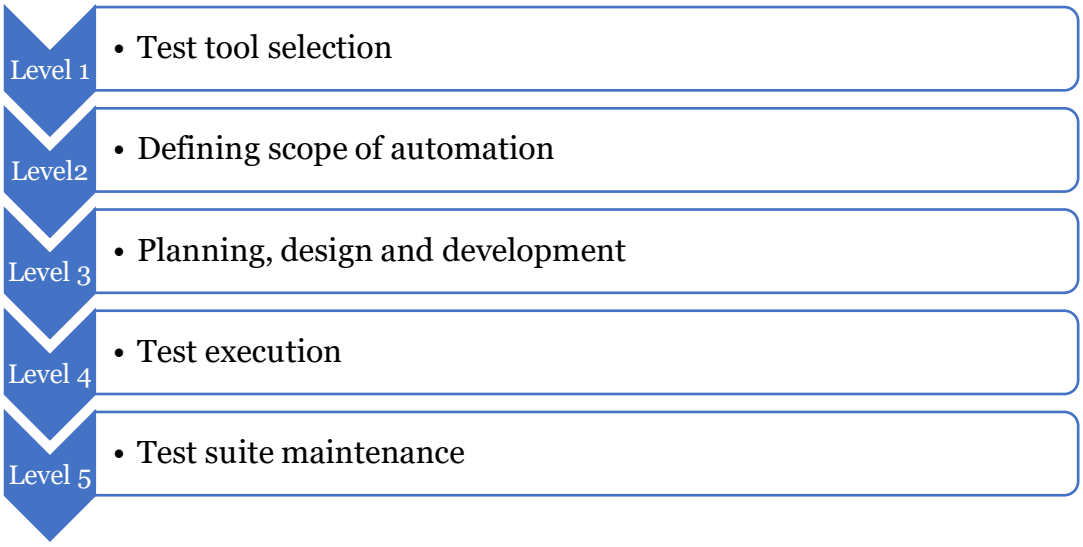


Figure 8: Levels of automation testing process.

2.4 Need and challenges of automation testing

Purpose of the automation testing is similar as manual testing or any other existing testing methods, which is to identify the problem in the software, application, or system under test, so that it can be fixed, and the product can be made robust and reliable for users. However, the problem arises when some simple or repetitive, but time-consuming tasks need to be regressively tested. Manual testing does not suffice the need in this scenario. Automation testing has proven very effective over the years in terms of time and cost efficiency. However, there are several critical aspects which needs to be managed properly, that can cause issues if ignored and leads to inefficiency in automation testing. These issues are discussed in this section.

To begin with, it is essential to consider the clear objectives for the automation process. If the tester starts the testing with the vague objectives like the tests just needs to be faster and better, then usually it ends with wasting time and efforts. Objectives should be more specific, realistic, and achievable. The goal is not just finding lots of bugs, nor to automate every test but to decide which tests are suitable for the automation and to support overall testing strategy [20].

The second issue that tester needs to consider is while selecting tools and frameworks. Choosing the best and expensive tools will not ensure success in

automation, instead the tools need to be carefully selected, maintained, and tailored according to the tests that tester needs [20].

Automation testing often faces several technical challenges, one key challenge addressed in this thesis is the level of abstraction and testware architecture. Testware architecture refers to the way in which all the testing related artifacts such as, test scripts, data files, images, libraries are organized. There are two common major abstraction layers: first is to separate the tests and second is to separate the tool or environment specific details from the testwares. Separation of tests refers to the separation between purpose of the test and implementation of the test using high level abstraction. Tester needs to describe the tests using high-level keywords. Separation of test execution tools from the testware includes designing a testware in such a way that it is independent of any platform or tool. In other words, the test logic, data, and test structure are not tightly coupled with the execution environment. Instead, a control layer or interpreter- a software that translates high-level test instructions into executable actions, can be used to serve as an intermediary between the testware and the execution tool. This approach helps make the testware design scalable and maintainable. An object map which is used to store references to UI elements, simplifies updates and improves test readability. Using wrappers around UI elements like buttons, widgets and graphs are also recommended to enhance flexibility and reusability [20].

In the rapidly changing HMI environment, using abstraction to separate tests from the underlying platform is highly beneficial. When the environment changes, then tester will only have to modify the low-level scripts related to tool, platform, or execution environment.

One other common technical issue is failure analysis- the process of identifying the cause when a test fails, which is also a time-consuming process. In manual testing, the tester usually knows what went wrong and at which step, making it easier to resolve the issue. On the other hand, in automation testing when a test fails, the tester often only knows the identity of the bug, without any detailed information, making it difficult to resolve the bug. To address this lack of context issue, automated tests should log the additional data during test execution. Tools that provide log reports and output summaries after the test execution are especially useful in such scenarios [20].

Choosing a coding language is also an important step, as not all testers know all the languages. Therefore, it is better to use a language as simple and easy to understand for writing test cases. This thesis promotes the use of clear and straightforward language in test case design.

For instance, in Robot Framework, test cases can be written using keywords such as:

1) *When*: To define a condition,
2) *Then*: To specify the expected outcome,
3) *Verify*: To check the result,
4) *Click*: To interact with an object,
5) *Show* or *Log*: To display data.

This keywords-driven approach allows individuals, even those without programming knowledge, to contribute to test development.

2.5 Test automation frameworks

Automated testing framework provides a structured set of guidelines, tools, and a reusable foundation to help automate the test modules. The factors to consider when choosing the right framework include the categories of framework, attributes, and the trade-offs associated with each. These aspects should be thoroughly explored before adopting a framework for application development [21]. In addition to framework attributes and compatibility, there are several other important considerations such as:

1) Long-term maintenance needs, ensuring the framework can adapt to dynamic project requirements.
2) Framework integration with integration/continuous deployment (CI/CD) pipelines.
3) Cost and licensing.

There are various test automation frameworks, each designed to address specific testing requirements and project needs. To select a suitable testing technique, it is important to understand the diverse types along with their advantages and disadvantages. The main types of testing techniques are described below:

1) *Linear scripting*:
   In this technique, test cases are simple and scripted in step-by-step manner without modularization. These linear automation scripts are created by record and playback tools, where user records a sequence of actions and then converts them into a list of commands, which can then be replayed to repeat the same actions. It does not require any prior knowledge about programming [12].

2) *Model-based testing*:
   This technique uses models, functions to segregate the test cases based on the component or functionality. This framework promotes the reusability of tests by allowing common behaviors to be defined once and reused across multiple tests. Maintenance becomes easier as

updates are localized within the modules, reducing the impact of changes on the overall test suites [12].

3) *Data-driven testing*:
In this technique, test logics are separated from test data, allowing the same script to run with multiple data sets [21]. These data sets are saved externally and can be in any data sheet formats like xls, xlsx, and csv files. This approach helps to reduce redundancy by allowing testers to validate various scenarios with various data sets. To setup this framework requires proficiency in multiple programming languages to manage the data parsing, test execution, and result validation efficiently.

4) *Keyword-driven testing*:
This technique uses the high-level keywords containing more detailed test instructions. These keywords are stored in a table or a file along with the test data and the expected results. It allows multiple users to design test cases at different level of abstraction [12]. Technical users can define the underlying keyword logic and non-technical users can create tests by combining keywords. This framework also reduces the time constraints for maintaining and modifying the tests, as only the keywords get modified rather than the whole test script.

5) *Behavior-driven development (BDD)*:
This technique uses the natural language (syntax as Given-When-Then) to write the test cases using scenarios to promote collaboration between technical and non-technical users [21].

6) *Hybrid testing:*
This technique helps user to write the test cases by combining features from all the other listed frameworks to leverage their strengths [21].

7) *Page object model (POM):*
This technique is widely used in web automation. It uses the encapsulation of web page elements and their interactions into reusable objects [21].

2.6 Various techniques of testing GUI

GUI testing means testing the system's application through its graphical user interface. GUI testing ensures that the graphical elements of an application function correctly and meet design specifications. Elements like buttons, menu, drop down action, icons, text boxes, widgets and the whole layout are the focus of the GUI testing. This testing is crucial for the seamless user experience and usability. It is needed to detect the inconsistencies, functional errors, and layout issues in the interface. GUI testing is challenging, fragile, time-consuming, and sensitive to changes in the layout due to the large number of possible ways of user interactions and execution paths. Since, manual testing checks only a small portion of the application's functionality given the complex interfaces, automated testing is needed to test the system more efficiently [23].

There are several types of GUI testing techniques, each has its strength and drawbacks. The most common ones are as follows:

1) *Capture/replay testing*: capture/replay testing is the state-of-the-art in GUI testing field. This testing involves tools that captures user interactions with the UI and develop the scripts to automatically replay those actions by recording them. There are many tools that can be used for capture/replay testing such as Selenium, QF-Test, Robot tester.

   The advantage of this technique is that it is very simple and easy. The tester only needs to execute the test case manually once and then record it to replay or automatically run later. However, there are some disadvantages of this technique, particularly its sensitivity to even the smallest changes in UI. The recorded script can easily break when the UI is modified, which leads tester to repeatedly fix or repair the script. As GUIs and requirements change often, this leads to test script maintenance challenges [23].

2) *Visual GUI testing (VGT)*: VGT technique is designed to overcome the challenges of other technique such as lack of robustness, maintenance issues, and sensitivity to UI changes. VGT has emerged as a promising technique in industrial practice to improve the quality of GUI testing. This technique is one of the available automation techniques for high-level testing. VGT uses image recognition to automate the tests which were initially performed manually. Few examples of VGT tools are SikuliX, Eggplant Functional, EyeAutomate and Applitools Eyes [23].

   Some of the VGT's advantages include improvement in the bugs finding ability over manual regression testing and it is easy to use even

without any prior programming language knowledge. However, VGT has its own drawbacks such as test maintenance related issue, tool immaturity, image recognition volatility and limitation in use for highly animated GUIs. Also, the execution speed of VGT scripts is slower than the other testing technique [23].

3) *Model-based testing (MBT)*: This model-based testing has become one of the most effective techniques for GUI testing in recent years. It has been proven very effective for addressing the limitations of manual and scripted testing, especially with the growing complexity of GUIs. MBT uses abstract models like state machines to represents the expected behaviour of the UI, and automatically generates the test cases based on these models. In a study, the authors used the MBT approach to the AHLTA-Mobile healthcare application to address the challenges of testing reactive systems. Their work involved creating high-level behavioural models using MBT tools like NModel (by Microsoft) to generate and execute test cases [31].

   Like many other testing techniques, MBT also comes with both benefits and limitations. In the study, the authors highlight several advantages of MBT, including [31]:
   1) Early test development, allowing tests to be designed before implementation.
   2) Improved test coverage, ensuring a broader range of system behaviours are tested.
   3) Automation, which reduces manual effort in test case generation.
   4) Support for regression testing, enabling consistent testing during software updates.

   However, the study also points out some key limitations:
   1) Difficulty in mapping GUI actions to model actions, which can complicate model creation.
   2) State explosion, due to the large number of possible GUI states and transitions.
   3) Tool limitations, as many MBT tools are not fully optimized for GUI-based systems.

2.7 Emerging trends: AI-driven and ML-enhanced testing strategies

Industries are increasingly incorporating AI and machine learning (ML) technologies into both product development and testing procedures. AI/ML are rapidly emerging as transformative tools in the field of automation testing, aiming to make it more efficient, adaptive, and less dependent on human efforts. Recently, there has been growing interest and exploration of AI/ML applications in, driven by their ability to maximize test coverage, optimize test execution, and intelligently detect defects [25]. This section summarizes current developments and outline the future scope of these trends in automation practices.

There are many features that are offered by AI/ML such as autonomous test generation, self-healing tests and scripts, natural language processing (NLP), visual testing by leveraging computer vision algorithms, integration of AI into development and operations (DevOps) and CI/CD pipelines, and predictive analytics. These features are briefly described below [25][26]:

1) *Test case generation*: This feature leverages ML to scan and analyse the data to identify patterns of codebase, supporting in comprehensive test cases generation. This approach enhances the test coverage by covering a wide range of scenarios.
2) *Self-healing tests and scripts*: This feature utilizes AI to automatically adapt and update the tests scripts in response to changes in the SUT. By using ML algorithms, the AI tools can detect and analyse modifications in system's behaviour, finding which test scripts are impacted and making the necessary adjustments. This ensures the alignment of scripts with the dynamic system behaviour.
3) *Natural language processing*: Integrating NLP concept into test automation tools is also trending and has the potential to enhance automated test development and maintenance. Writing test scenarios in natural language commands and converting them into executable test scripts by using AI-powered tools can bridge the gap between technical and non-technical teams.
4) *Visual testing*: This approach involves automated detection and verification of visual elements by leveraging computer vision algorithms. This provides AI systems to analyse visual elements of an application by comparison with baseline image screenshots, identifying potential visual discrepancies.
5) *Integration with DevOps and CI/CD pipelines*: As agile practices become more widespread in organizations, integration of AI into DevOps and CI/CD workflows has become an essential feature for ensuring the maintenance and alignment of tests with the continuous development and release cycle.

Several obstacles in the current AI/ML driven testing practices exist such as [25][26]:

1) Data quality for training AI algorithms: Inadequate or biased data can lead to poor performance and unreliable test results.
2) Integration challenges with the existing testing frameworks and workflows- Incorporating AI-driven testing into established testing environments can be complex and may require significant customization and architectural changes.
3) Skill gap in AI/ML testing and data analytics: Many teams often lack the expertise in AI/ML and data science that can hinder effective implementation of AI-based testing solutions.
4) Security and privacy concerns for using sensitive data for training AI models.
5) Tools complexity- complex AI-powered AI tools can lead to difficulties in their configuration, maintenance, and optimization.
6) Algorithmic biases in the algorithm can introduce distortions in test results or make wrong predictions based on the training data and can overlook certain types of defects effecting reliability of automated testing.

These problems need to be carefully considered and addressed to establish a successful and reliable AI-driven testing process. These issues can be mitigated by making some strategic improvements, for instance, establishing robust data management practices to ensure the data quality and integrity, upskilling teams to bridge the knowledge gap in AI/ML. Simplifying and optimizing AI tools can also reduce configuration and maintenance issues, while adopting diverse datasets to mitigate algorithm bias. Security and privacy concerns can be addressed by applying data encryption and strict action controls [26].

3 Prerequisites for HMI testing

3.1 Description for test setup

The HMI testing environment is designed to evaluate the three distinct displays. Two of these displays are connected to the tabletop hardware setup showed in figure 9. Figure 10 shows the third display, which is a web-based UI connected to an HIL simulation.

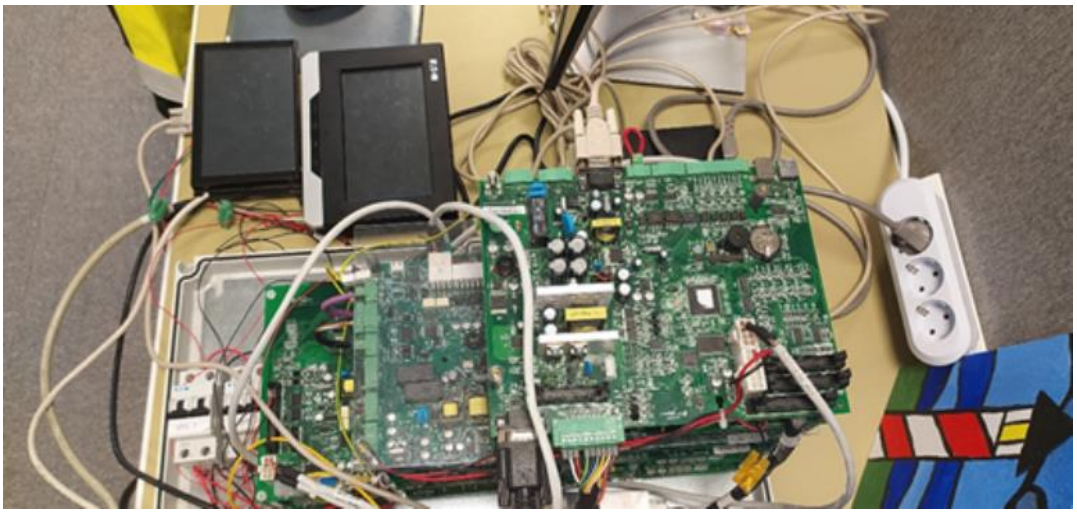


Figure 9: Tabletop hardware setup integrated with displays for HMI testing. Image captured by the author during testing.

The hardware setup in figure 9 includes all three interface boards (i.e. MCU, UPM and CSB) stacked together and connected to each other. The HMI connects to the setup through the communication port on the CSB board. The user's personal computer (PC) is also connected: it links to the HMI via an Ethernet cable and to the CSB via a recommended standard 232 (RS-232) to universal serial bus (USB) adapter. User's PC contains all the tests written in the Robot Framework and sends commands to the hardware components, both the CSB board and the HMI, to perform required operations.

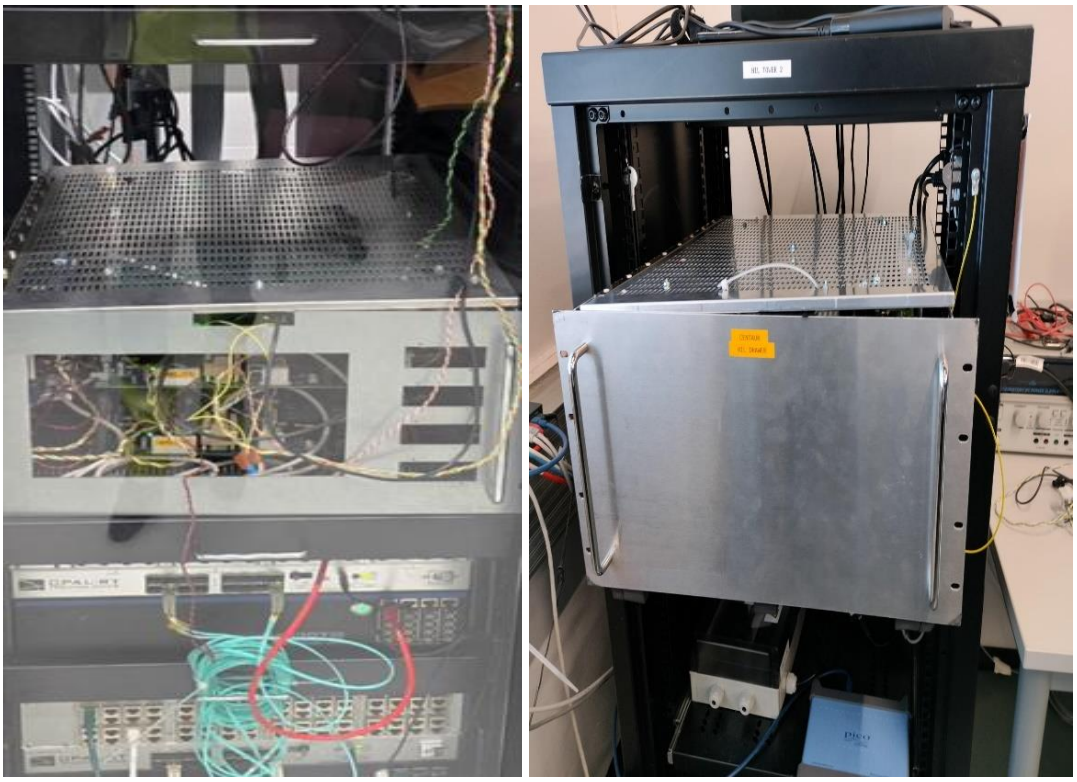


Figure 10: Hardware-in-loop setup integrated with web-UI for HMI testing purposes. Image captured in the lab environment by the author and a team member Prajjwol Tripathi.

HIL is a simulation setup where real controller hardware is connected to a real-time simulation of the system it controls. HIL testing is a technique that places an embedded system into a simulated environment that mimics real-world operating conditions, eliminating the need to connect to the actual physical system. The setup has OPAL-RT's real-time digital simulator. It features a powerful target computer, a high-end configurable field- programmable gate array (FPGA), signal conditioning for up to 256 input/output (I/O) channels, and 16 fast fibres optic small form-factor pluggable (SFP) interfaces. A separate PC is used as the user workstation which runs MATLAB/Simulink to simulate the UPS states and uses RT-LAB software to connect the simulation to the HIL hardware in real time [12].

The setup includes two controllers: MCU and UPM. The UPM handles the control of the rectifier, inverter, and the battery converter. The MCU oversees all the UPMs and maintains the controller area network (CAN) communication between the modules. The controller is physically connected to both the HIL I/O modules and the user workstation. User can run tests on the user workstation, and HIL system will simulate those operations and presents the results on the web-UI interface. To run automated UI tests, the user needs to

install the Robot Framework along with all its required libraries and dependencies.

3.2 Test automation scope

Defining scope for the automation testing is a major step. This step enhances efficiency of the tests. It is crucial to define what tests should or shouldn't be automated. Key points that need to be considered before selecting the test cases are as follows:

- Should automate- frequently repeated tests, difficult or time consuming to run manually, tests with clear pass/fail criteria, and tests based on visual perception that can be reliably validated through automation.
- Shouldn't automate- tests without evident pass/fail results, features with changing requirement, tests that will be executed only once, and tests involving raw or unstable functionality.

The scope of this thesis is designed by considering above mentioned key points. In testing, two major categories are commonly considered: functional testing and performance testing. This thesis focuses only on the functional testing of the UPS and does not cover the performance testing. Functional testing is a type of testing that verifies whether the system behaves as intended, based on its requirements. It focuses on what the system does, and not how it does it. Whereas performance testing checks how well a system performs under different and extreme conditions. Performance testing focuses on things like speed, responsiveness, stability, and resource usage when the system is being assessed under stress.

These tests were designed to simulate and verify the basic behavior of the UPS system on the HMI. The scope includes:

1) Simulation of UPS states such as online mode, bypass mode, battery mode, and load off state.
2) Verification of HMI mimic for all the modes mentioned above.
3) Interaction with the HMI elements such as buttons, icons, and menus to ensure the correct response of the elements.
4) Validation of date and time displayed on the HMI to confirm correct synchronization and formatting.
5) Reading and verifying meter values such as voltage, current, battery percentage, and load.

3.3 Limitations

In the context of this thesis, several limitations were observed during testing. While the automation framework provided significant improvement in efficiency and scalability, these limitations introduced certain constraints that affected test execution speed, performance, and reliability. Some HMI screens displayed delayed responses or slow content rendering. This issue was resolved by introducing extra buffer time (e.g., sleep intervals) after certain test steps. However, this approach increased overall test execution time. At the current stage of development, the test framework does not support microsecond-level timing operations, which limits the ability to validate time-sensitive operations or transitions in the HMI. This test framework is also sensitive to network latency since tests execution relies on remote access of the screen's UI. This can affect the responsiveness and reliability of automated tests.

To support validation of HMI screens, this thesis explored the use of optical character recognition (OCR) via the pytesseract library, a Python wrapper for the tesseract OCR engine. It is an open-source OCR engine which was originally developed by Hewlett-Packard and later maintained by Google in 2005 [27]. Tesseract is used for extracting texts from images and scanned documents. In the context of this thesis, it is utilized for automating validation by recognizing and extracting textual content from HMI screenshots, enabling test automation where direct access to UI element is limited. While effective in many cases, the tool occasionally skipped or misread text elements. This was improved by using higher configuration of the tool and preprocessing techniques. However, the tool's accuracy is dependent on the image quality: low resolution, poor contrast or noise can lead to incomplete or inaccurate text extraction.

3.4 Reusability of test cases

Writing automated test cases alone is not sufficient; ensuring their reusability is equally critical. Reusable test cases save time and effort by allowing testers to adapt to dynamic changes without having to recreate tests from scratch. Test case creation should prioritize reusability, with careful consideration of how broadly and frequently each case can be applied across evolving system conditions. There are four factors that can be taken into consideration while generating these reusable test cases. These factors are understandability, changeability, independence, and universal [28].

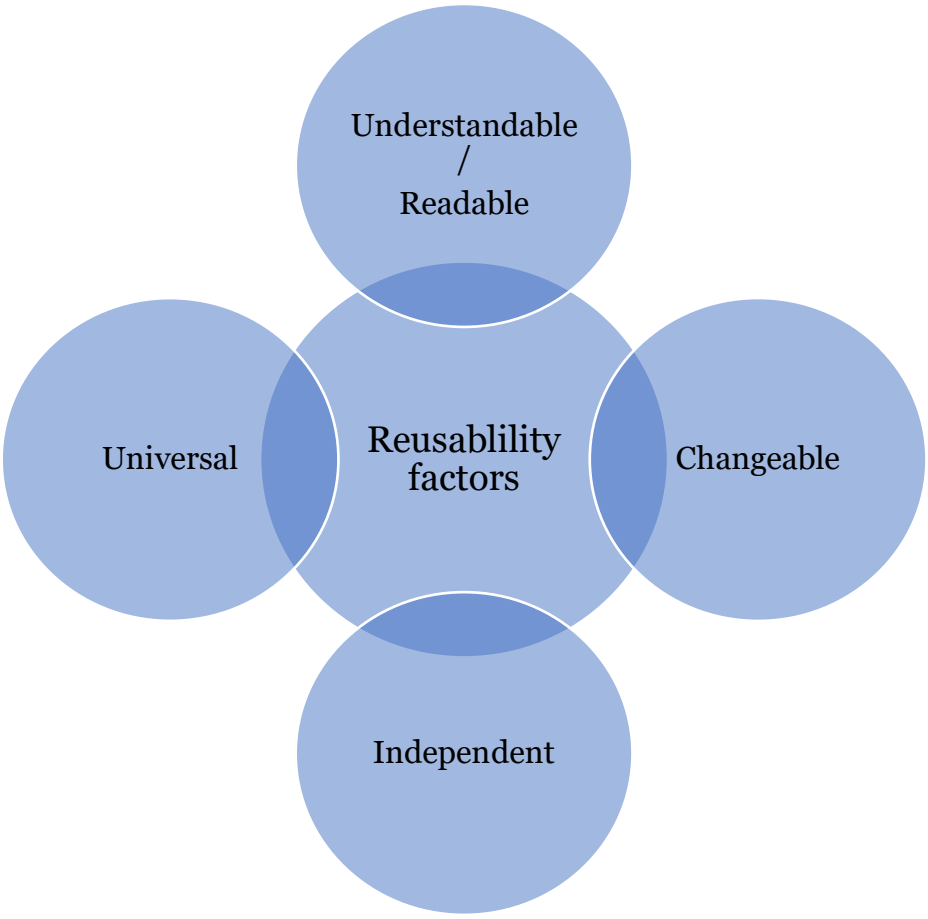


Figure 11: Core attributes of reusable test cases.

Understandability refers the ease of understanding the test case based on its description such as keywords or test case summary. Changeability refers to how easy a test case can be modified to accommodate updates of the system under test. Changeability can be achieved by introducing the structure and style of test cases that allow for easy and consistent modifications. To achieve higher changeability; it is preferable to have less constant and more variable elements in test case design. Independence refers to the ability of a test case

to function without relying on the execution or outcomes of other tests. Universality, on the other hand, reflects how broadly a test case can be applied across different modules, platforms, or configurations, enhancing its long-term value, and reducing redundancy [28].

Abstraction testing technique has been proven efficient over traditional concrete test cases. Abstract test cases are less dependent on specific details, making them easily adaptable and reusable across multiple configurations of the system. This technique reduces the size of code by making test reusable while maintaining the coverage [29]. This thesis adopts the abstraction-based testing approach to make scalable tests for HMIs. Instead of solely relying on concrete, implementation-specific test cases, abstract cases were designed based on functional requirements. For example, instead of writing concrete test like: “Enter 6-digit password to access the third level” an abstract test case would be “Enter password to access the third level”. This abstract test does not hardcode specific values like 6-digit, making it adaptable for different display configurations. Figure 12 and 13 shows different configurations for example test case explained above.

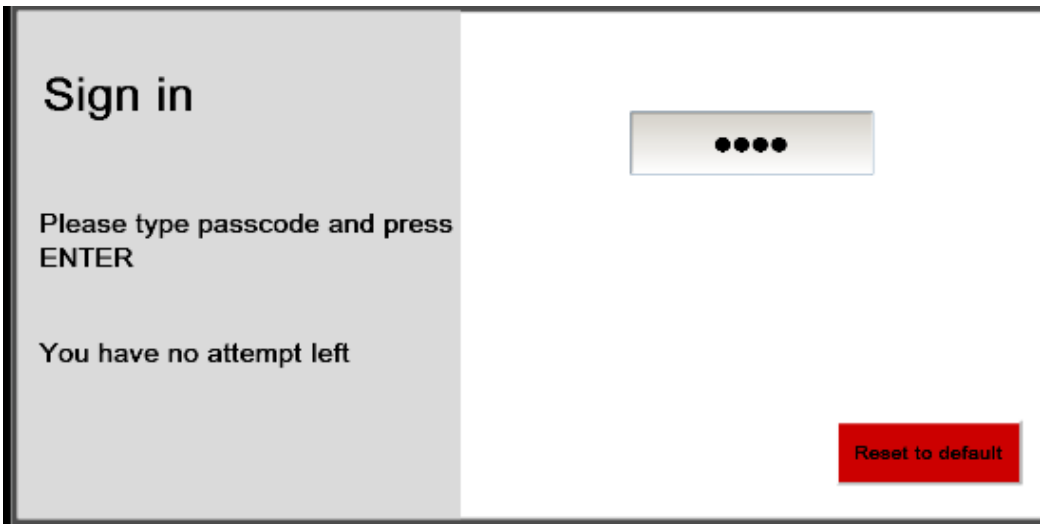


Figure 12: Self-documented Eaton HMI configuration interface with 4-digit password.

Figure 13: Self-documented Eaton HMI configuration interface with 6-digit password.

These high-level abstract tests cases can be adapted across different HMI configuration and firmware versions with minimal modifications. It ensures that the test logic remains consistent even as the underlying system evolves, making the tests more robust for future adaptation.

4 Testing frameworks and methods

A structured and systematic approach is essential for validating the quality of firmware in a system. This section presents the methods, tools and workflow used in the thesis. It also includes brief description of additional methods and tools that are essential for testing but were not utilized in the scope of this thesis.

4.1 Tools and frameworks

This section outlines the used tools and framework for functional testing.

- Functional Testing Tools
  Functional testing is required for validating the operations of a system. There are many tools available for testing the functionality such as Robot Framework, Google test, Tabletop simulation, Selenium, Cypress, Appium. However, Robot Framework is the primary tool that is used in this work. It is an opensource and Python-based automation framework and it supports keyword driven testing. This framework provides users to write the test cases in a simple readable format. It generates detailed logs and output reports of executed test cases including information such as name and number of pass and failed cases, execution time and state of the execution. The following reasons were considered for choosing the Robot Framework for this thesis:

  1) *Keyword driven framework*- It allows creation of simple and easy to maintain test cases.
  2) *Versatile testing support*- It is suitable for general purpose testing, such as web, mobile and application programming interfaces (APIs) automation.
  3) *Browser support*- It provides multiple browser support via selenium or other libraries.
  4) *Platform independent*- It works across multiple operating systems without compatibility issues.
  5) *High extensibility*- It allows creation of custom libraries to extend the functionality as per requirements.

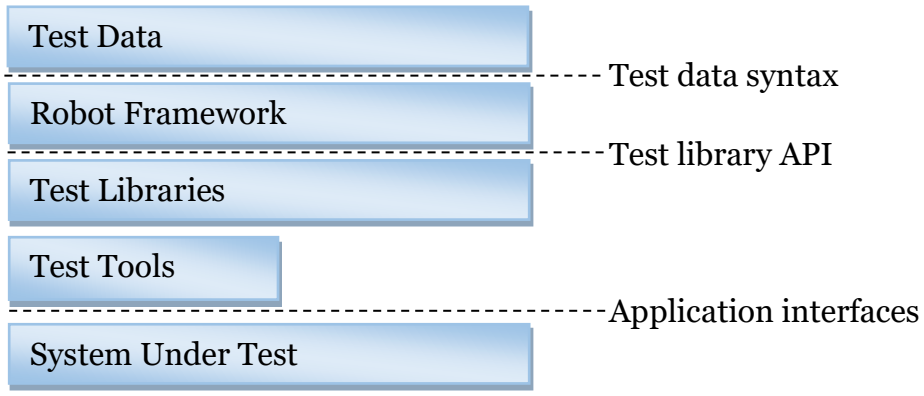


Figure 14: Robot Framework architecture referenced from Robot Framework user guide [30].

Figure 14 shows the high-level overview of the Robot Framework's architecture. The first layer is the test data layer. This layer is written in the Robot Framework test data syntax (i.e., keywords) which acts as an input for the Robot Framework to process. The second layer is Robot Framework core which executes keywords through the test library API. The third layer consists of test libraries (standard or external such as Selenium/Appium) and supporting test tools. These libraries handle the interaction between Robot Framework and system under test. These libraries can communicate directly with application interface (e.g., REST, Modbus, WebDriver) or can use lower-level test tools as drivers. Finally, the last layer is system under test, on which the operations are performed [30].

Other tools such as performance testing tools and tests management tools were not utilized in this thesis; however, they are potential options for future enhancements of the developed test suite.

- *Test Management Tools*
  The right test management tool is required to make software test automation more efficient. In a development environment characterized by continuous project evolution, agile methodologies, and continuous integration practices, a robust and adaptable tool can significantly improve the testing workflows and overall software quality. Among the many test management tools such as TestRail, Xray, PractiTest and Testmo available in the market, Zephyr stands out as a widely adopted solution.

  Zephyr: Despite the availability of various tools for test management, zephyr caters to teams working in agile environments. It offers seamless integration with Jira, allowing users to manage test cases, execute test cycles, and track results directly with their existing project

workflows. Zephyr supports both manual and automated testing, making it a flexible choice for various teams. Its real time reporting features and traceability capabilities help enhance the software quality assurance process [31].

- *Performance Testing Tools*
  Performance testing is done to evaluate the system's speed, responsiveness, reliability, scalability, and resource usage under various conditions [32]. In the context of UI performance testing, it involves assessing how the UI behaves under various stress conditions and operational load. To evaluate the performance, key metrics include startup time, screen loading time, response time to user input, data refresh rate, memory, and central processing unit (CPU) usage, and communication delays.

  Various performance testing tools such as Apache JMeter, Grinder, HttpRider, Locust, and Gatling are commonly used and are suitable to evaluate the performance of web-based UI. However, app-based UI performance testing requires different approaches such as custom scripts, diagnostic reports, or profiling tools supported by the HMI platform [32].

4.2 Testing Methods

Several methods were used in designing the test cases for this thesis work, including error seeding technique, input partition testing, and equivalence classes technique. In addition, boundary value analysis is proposed as a complementary method to further enhance the coverage, effectiveness, and completeness of the test suites. These methods are briefly explained below to provide context for their application in the testing process.

- *Error seeding*:
  Error seeding technique involves intentionally inserting known errors into the code to verify if the testing process can detect them [33]. This method helps evaluate the effectiveness of the test cases and accuracy of bug reporting. In this thesis, this technique was employed to verify the correct date and time shown on the display. The HMI was programmed with firmware containing incorrect date and time values. The corresponding test case failed as expected and successfully reported wrong date and time, hence confirming the reliability of the test case in reporting the bug. There can be more similar scenarios where this method can be applied such as inserting wrong icon, disabling a button, or showing incorrect status.

- *Input partition testing*:
  Input partition testing technique involves dividing input data into partitions or groups where test cases are designed for each group [33]. It reduces the number of test cases while maintaining good coverage. In this thesis, input partitioning was applied to user interaction elements on the HMI. Specifically, clickable components were segregated into two groups: buttons/tabs and Icon elements. One set of test cases included the clicking of the buttons/tabs and other test case was designed for clicking and validating the response of these icon elements.

- *Equivalence classes*:
  Equivalence class technique groups inputs into equivalence classes, where each member is expected to produce similar outcomes [33]. This approach also reduces the number of test cases while maintaining the coverage. It assumes that if a test case passed for one representative input from a class, the other inputs in the same class are likely to behave similarly.

  For example, when testing different date and time formats on an HMI, all formats that are considered valid (e.g., DD/MM/YYYY, MM-DD-YYYY, YYYY-MM-DD) can be grouped together into one equivalence class. If the system correctly handles one format from this class, it is expected to handle the others similarly.

  Another example of this technique was used in testing the UPS state transitions. States such as online, shutdown, bypass, and battery state were grouped into a single equivalence class representing UPS operational states. During testing, it was verified that commanding transitions between them resulted in correct and expected state changes.

- *Boundary value analysis*:
  Boundary value analysis is a technique that focuses on the testing the values at the edges of input domains [33]. Errors often occur at the boundaries rather than in the input range. This technique is particularly useful in scenarios involving meter readings such as load percentage, battery voltage, and temperature thresholds. For example: if the acceptable battery voltage range is between 200 V and 240 V, test case should be able to test with the 199 V (edge value), 200 V (lower limit), 240 V (upper limit), and 241 V (edge value).

4.3 Workflow and test architecture

The HMI test automation was carried out using Robot Framework on a local PC, which was connected to the hardware setup or HIL setup. The test scripts were executed on the PC in Robot Framework, initiating communication with the UPS hardware via the configured protocol. Once the communication gets established, Robot Framework sends the test instructions to the UPS hardware, which then relays the commands to the HMI. The HMI responds by performing the specified actions, such as simulating button clicks, loading meter values, and switching screens. Robot Framework captures the results and validates them against the expected outcomes. After execution, a detailed log report gets generated in Robot Framework for further analysis. The complete test procedure is shown in figure 15.

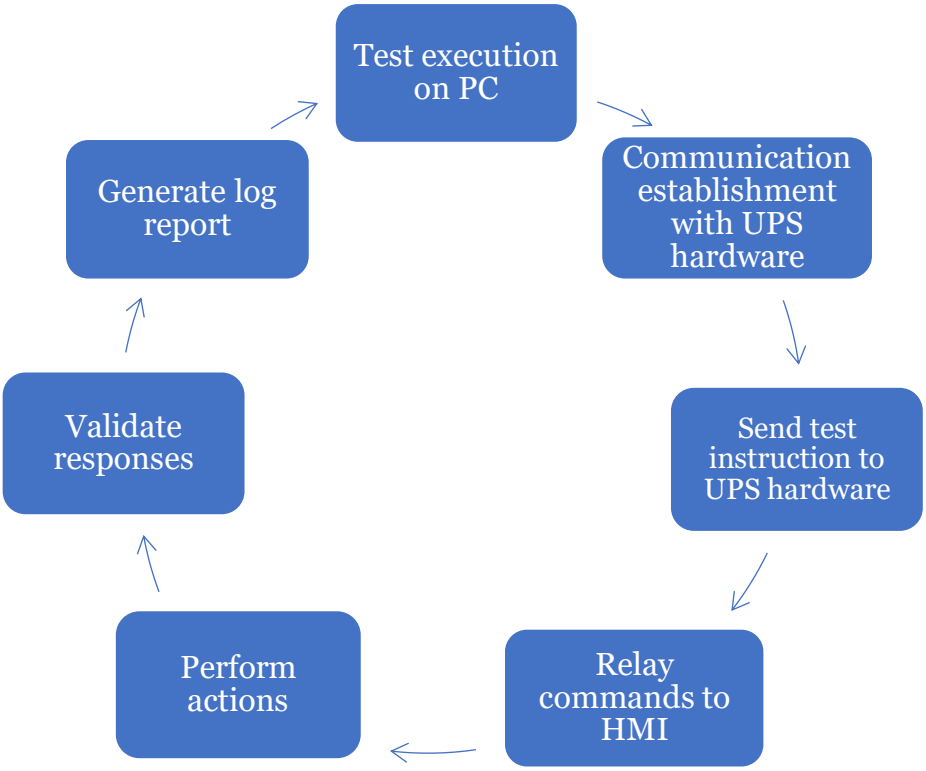


Figure 15: Test procedure for the HMI automation testing.

The HMI test automation framework follows a layered architecture that utilizes high-level abstraction. This structure separates generic testing logic from product-specific implementations, enabling the reuse of test cases across different UPS model and HMI configurations. Figure 16 represents the layered architecture which is implemented for this thesis work.

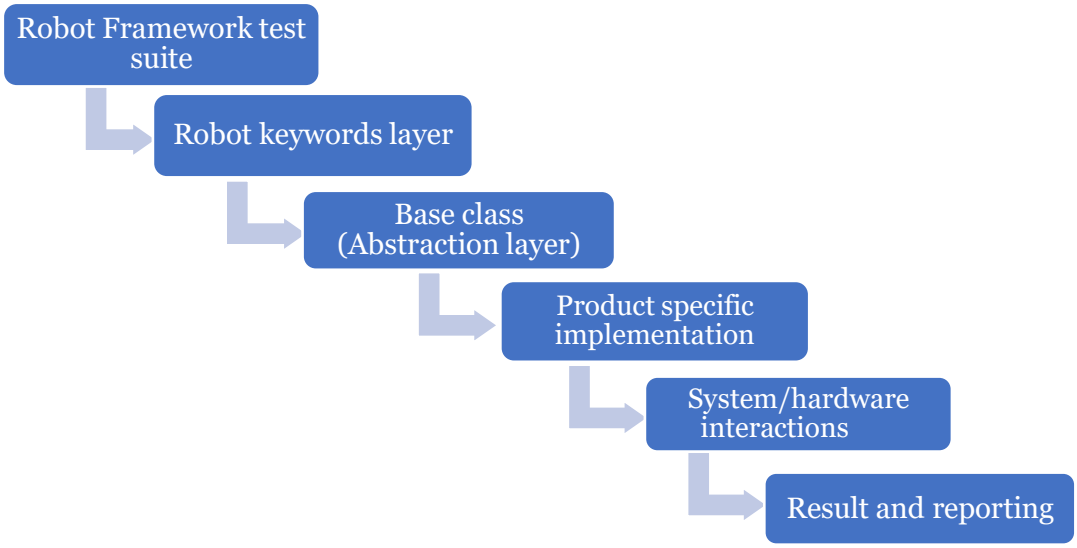


Figure 16: End-to-end test execution flow.

The top-level layer is tests suite layer, which contains high-level test cases, and executes these test cases. These test cases are written using keyword-driven syntax for readability and maintainability. Each tests represent a functional part of the HMI such as displaying UPS state transitions, alarm updates, meters values representations, correct working of HMI elements such as buttons, icons, widgets, and many other UPS details.

The second layer is the keywords layer, which acts as a bridge between high-level tests cases and the underlying logic implementations. These keywords encapsulate the functions and internally calls the methods from the base or product specific implementation layer.

The third layer is the base class or abstraction class layer. This layer is responsible for setting up and tearing down the test environment for execution of tests. This layer also acts a delegation point, forwarding all the method calls from keywords layer to the product specific implementations layer.

The fourth layer is named as product-specific implementations layer as it contains implementations tailored to a specific UPS model or HMI variant. This layer receives all the delegated calls from base classes and implement hardware specific communication protocols for the specific UPS product model.

The fifth layer is system or hardware interactions layer. This layer manages direct communications with the physical UPS hardware via communication protocols like Modbus, REST API, and virtual network computing (VNC)

connection. This layer ensures accurate data exchange during test execution using communication protocols.

The sixth layer is responsible for collecting the result of each layer's operations and aggregating all the outcomes of the whole test. This layer generates a comprehensive test report with all the statistics such as test names, pass/fail status, execution duration and logs with timestamps.

5 Results

This chapter presents the outcomes of the test cases designed and executed during this thesis work. The results are organized by test groups in accordance with the scope of this thesis mentioned in section 3.2. The discussion begins by test case names, followed by outlining the logic behind each test group and their execution results.

5.1 Test case overview

This section provides a concise summary of individual test cases highlighting the test case identifier and discusses the implemented logic behind each. This section serves as a reference point for the detailed results analysis provided in the next section.

Test description (Group 1):
Group one includes the transitions between different UPS states such as online mode, bypass mode, battery mode, and shutdown mode or load off state, followed by the verification of internal state and HMI mimic. HMI mimic shows the correct mimic, energy flow indication, and each component's state.

Test case name:
Basic UPS operational tests.

Logic description:
Figure 17 illustrates the logic flow for the given test, showing the validation process of UPS operational states.

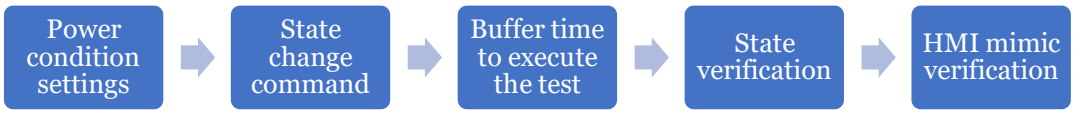


Figure 17: Logic flow diagram for group one test.

First, power conditions get configured to prepare the system for testing. Then, a state change command is executed, triggering the contactors to route power through the appropriate paths. The system transitions to the target state, which is verified internally. Finally, the HMI mimic is verified to confirm the correct visual representation of the state.

Conclusion: The test case passed successfully. It was executed and verified on both web UI and display hardware confirming consistent behavior across platforms.

Test description (Group 2):
Group two test involves verifying the synchronization between display's date and time and real-world system time. Thes test retrieves the HMI time, formats it, and compares it with the current PC time. This test is design to evaluate the test's capability to detect mismatches or bugs, confirming its effectiveness in identifying synchronization issues.

Test case name:
Verify HMI date and time matches with real-time world clock.

Logic description:
The test logic flow shown in figure 18 illustrates the process of validating the date and time formats.

Figure 18: Logic flow diagram for group two test.

The test begins by sending a command to retrieve the HMI's date and time. The response gets processed to manage multiple formats. Then, the current system's date and time is fetched and gets compared with the HMI's processed time string to verify synchronization.

Conclusion: The test case failed, successfully detecting the intended bug. This failure confirms the test's ability to detect and report the targeted bug scenario.

Test description (Group 3):
Group 3 tests involve testing user interaction with HMI element like tabs or buttons and verifying the responsiveness of these elements. The test ensures that the buttons are responsive, and triggers correct page loading.

Test case name:
Verify buttons are clickable on home page.

Logic description:
Figure 19 illustrates the logic flow for the given test, which focuses on the responsiveness of buttons.

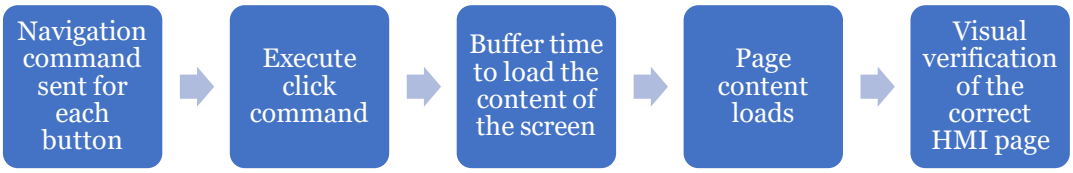


Figure 19: Logic flow diagram for group three test.

In UPS product 1, a referenced image of the home page is used to calculate button coordinates for navigation. In UPS product 2, navigation is performed by using predefined web-UI element IDs for each button. After executing navigation command, a click command is sent, followed by a short buffer time to allow the respective page content to load.

Conclusion: The test case passed successfully, confirming that all the buttons on the home page of both screens are clickable and correctly loads the intended page.

Test description (Group 4):
Group four involves testing user interaction with icons such as the lock icon and verifying loading of the correct screen. The test ensures that clicking the icon triggers the expected navigation and content display.

Test case name:
Verify lock icon clicks and shows sign-in page.

Logic description:
The logic flow for this test is illustrated in figure 20, which follows a similar structure to figure 19, as the test logic in group four closely resembles to group three.

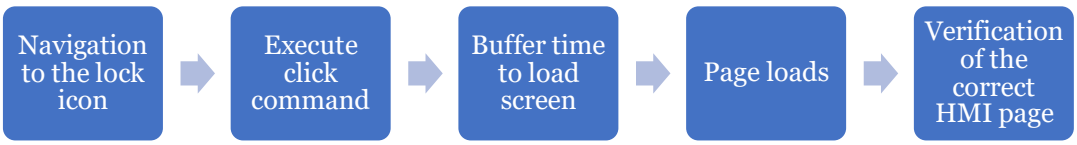


Figure 20: Logic flow diagram for group four test.

In UPS product 1, lock icon's coordinates are calculated for the navigation, while in UPS product 2, element ID is used. After navigation to the icon, a click command is executed to verify icon's responsiveness. A short buffer

time is provided to allow the page to load, followed by an image comparison between expected and actual screens to ensure that the sign-in page has loaded correctly.

Conclusion: The test case passed as intended, the test verifies that the icon is responsive and loads the correct screen.

Test description (Group 5):
Group five involves the reading action from the HMI. This test is performed to read the meter values such as load value, efficiency percentage, battery percentage, output, estimated runtime, and input consumption from the UI.

Test case name:
Read home screen meter values.

Logic description:
Figure 21 presents the logic flow for the group five test, which focuses on reading and validating the meter values from the UPS system.

Figure 21: Logic flow diagram for group five test.

For UPS product 1, meter values are retrieved by reading the corresponding Modbus registers where the meter data are stored. In UPS product 2, values are accessed by the class name of web-UI elements. After navigation to the appropriate locations, the values are extracted, units are formatted, and the data is validated to ensure it is not empty. Finally, the values are represented in a standardized and readable format.

Conclusion: This test case passed successfully. All expected meter values were retrieved and displayed in a readable format.

5.2 Result analysis

This section discusses about the outcomes and findings of the practical test execution. It also discusses about the various approaches that were used during the test execution and explains how the results were interpretated using log reports generated by Robot Framework.

One of the main challenges was to access the HMI elements in the app-based display setup. To overcome this, a remote connection was established using VNC viewer tool. Once the connection was established, OCR was employed to find and extract the texts from the HMI screen. The OCR output was then used to calculate the coordinates of UI elements, enabling simulated click operations on the UI.

After successful test execution, Robot Framework generated three key output files: output.xml, log.html, and report.html. Output file is in .xml format and it contains detailed execution logs, including tests steps, keywords, and results. Log file is in .html format which also contains detailed logs of test execution, keywords level details, and error messages. Similarly, report file is also in .html format which contains high-level summary of test execution with statistics and pass/fail status of test cases. These files were used to analyse any failures or unexpected behaviours. Figure 22 and 23 presents summery view from Robot Framework's report file illustrating the outcomes of automated test cases. Figure 22 shows a successfully passed test case, indicated by a green background and accompanying test statistics. Furthermore, Figure 23 highlights a failed test case, marked with a red background and corresponding failure details.

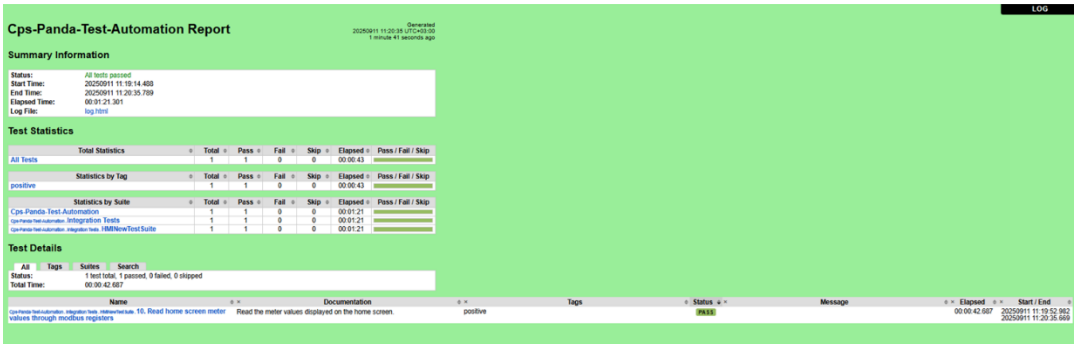
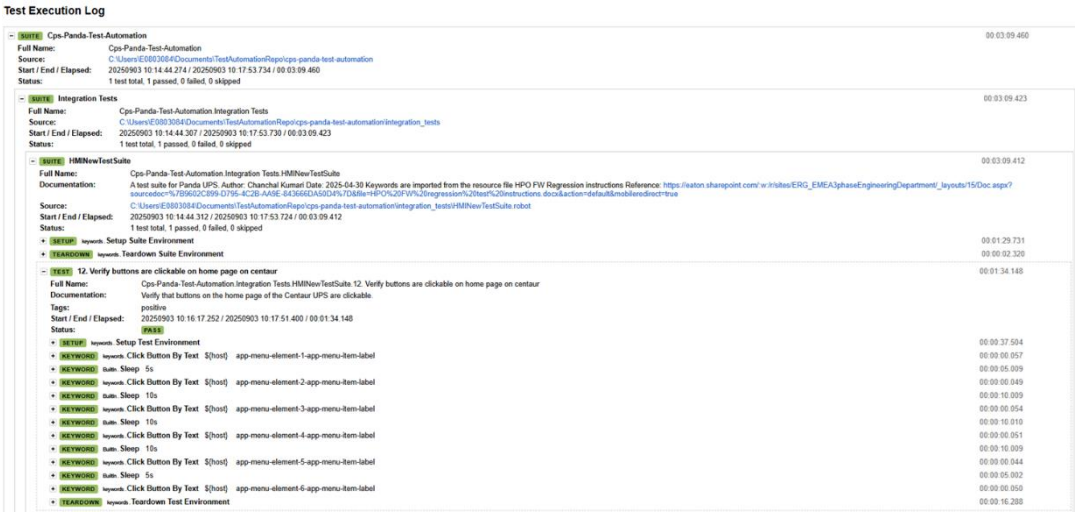


Figure 22: Summery view from Robot Framework's report file highlighting a passed test case.

Figure 23: Summery view from Robot Framework's report file highlighting a failed test case.



Test Execution Log

SUITE Cps-Panda-Test-Automation 00:03:09.460
Full Name: Cps-Panda-Test-Automation
Source: C:\Users\E0803084\Documents\TestAutomationRepo\cps-panda-test-automation
Start / End / Elapsed: 20250903 10:14:44.274 / 20250903 10:17:53.734 / 00:03:09.460
Status: 1 test total, 1 passed, 0 failed, 0 skipped

SUITE Integration Tests 00:03:09.423
Full Name: Cps-Panda-Test-Automation.Integration Tests
Source: C:\Users\E0803084\Documents\TestAutomationRepo\cps-panda-test-automation\integration_tests
Start / End / Elapsed: 20250903 10:14:44.307 / 20250903 10:17:53.730 / 00:03:09.423
Status: 1 test total, 1 passed, 0 failed, 0 skipped

SUITE HMINewTestSuite 00:03:09.412
Full Name: Cps-Panda-Test-Automation.Integration Tests.HMINewTestSuite
Documentation: A test suite for Panda UPS. Author: Chanchal Kumar Date: 2025-04-30 Keywords are imported from the resource file HPO FW Regression instructions Reference: https://eaton.sharepoint.com/:w:/r/sites/ERG_EMEA3phaseEngineeringDepartment/_layouts/15/Doc.aspx?sourcedoc=%7B9602C899-D795-4C2B-AA9E-84366EDA5004%7D&file=HPO%20FW%20regression%20test%20instructions.docx&action=default&mobileredirect=true
Source: C:\Users\E0803084\Documents\TestAutomationRepo\cps-panda-test-automation\integration_tests\HMINewTestSuite.robot
Start / End / Elapsed: 20250903 10:14:44.312 / 20250903 10:17:53.724 / 00:03:09.412
Status: 1 test total, 1 passed, 0 failed, 0 skipped
+ SETUP keyword Setup Suite Environment 00:01:29.731
+ TEARDOWN keyword Teardown Suite Environment 00:00:02.320

TEST 12. Verify buttons are clickable on home page on centaur 00:01:34.148
Full Name: Cps-Panda-Test-Automation.Integration Tests.HMINewTestSuite.12. Verify buttons are clickable on home page on centaur
Documentation: Verify that buttons on the home page of the Centaur UPS are clickable.
Tags: positive
Start / End / Elapsed: 20250903 10:16:17.252 / 20250903 10:17:51.400 / 00:01:34.148
Status: PASS
+ SETUP keyword Setup Test Environment 00:00:37.504
+ KEYWORD keyword Click Button By Text ${host} app-menu-element-1-app-menu-item-label 00:00:00.057
+ KEYWORD Built-In Sleep 5s 00:00:05.009
+ KEYWORD keyword Click Button By Text ${host} app-menu-element-2-app-menu-item-label 00:00:00.049
+ KEYWORD Built-In Sleep 10s 00:00:10.009
+ KEYWORD keyword Click Button By Text ${host} app-menu-element-3-app-menu-item-label 00:00:00.054
+ KEYWORD Built-In Sleep 10s 00:00:10.010
+ KEYWORD keyword Click Button By Text ${host} app-menu-element-4-app-menu-item-label 00:00:00.051
+ KEYWORD Built-In Sleep 10s 00:00:10.009
+ KEYWORD keyword Click Button By Text ${host} app-menu-element-5-app-menu-item-label 00:00:00.044
+ KEYWORD Built-In Sleep 5s 00:00:05.002
+ KEYWORD keyword Click Button By Text ${host} app-menu-element-6-app-menu-item-label 00:00:00.050
+ TEARDOWN keyword Teardown Test Environment 00:00:16.288

Figure 24: Detailed view of Robot Framework's test execution log displaying step-by-step actions along with their corresponding execution time and status.

Figure 24 presents Robot Framework's test execution log file, which provides a detailed breakdown of individual test steps. Each step is displayed together with its corresponding execution time and status, offering insight into the flow and performance of the test case. These details are valuable for debugging and performance analysis purposes.

Table 1: Test results description for the app-based HMI conducted on table-top setup.

Test Group	Test Status	Total test execution time	Message
Group 1 Basic UPS operational tests	Pass	00:03:44.283	--
Group 2 Verify HMI date and time matches with real-time world clock.	Fail	00:00:41.802	Date on the HMI does not match the current system date.: 2023-08-05 03:38:20! = 2025-09-11 09:56:05.0118
Group 3 Verify buttons are clickable on home page.	Pass	00:02:44.825	--
Group 4 Verify Lock icon clicks and shows Sign-in page.	Pass	00:00:47.531	--
Group 5 Read home screen meter values.	Pass	00.00.42.687	--

Table 2: Test results description for the web-based UI conducted on HIL setup.

Test Group	Test Status	Total test execution time	Message
Group 1 Basic UPS operational tests	Pass	00:06:43.431	--
Group 2 Verify HMI date and time matches with real-time world clock.	Fail	00:00:53.915	Date on the HMI does not match the current system date.: 2025-04-23 09:47:59! = 2025-09-03 09:49:58.432
Group 3 Verify buttons are clickable on home page.	Pass	00:01:34.148	--
Group 4 Verify Lock icon clicks and shows Sign-in page.	Pass	00:00:53.859	--
Group 5 Read home screen meter values.	Pass	00.00.42.687	--

Tables 1 and 2 present the executed test results on both HMIs, highlighting the key aspects such as test name, status, execution time, and any associated error messages. All result parameters are consistent across both HMIs, except the execution time. It varies for each test case in both HMI due to differences in their UI design architectures. For instance, in web-based UI, the home screen contains lesser number of tabs compared to the app-based UI, resulting in shorter execution times for certain test cases.

Conversely, some tests take longer to execute on the web UI not only because a new web browser instance is launched at the beginning of each test, but also due to potential differences in the underlying test logic tailored for each HMI. For example, some UI interactions or navigation flows may involve additional steps in one HMI compared to the other, affecting the overall execution time.

6 Conclusions

This chapter concludes the work carried out in this thesis, which focused on developing a unified solution for automated HMI testing that can be reused across various HMIs of Eaton UPS products. The automated test suite was validated on two distinct HMI setups: one featuring an app-based UI connected with a tabletop setup, and the other utilizes a web-based UI integrated with an HIL system. These setups differed in both design architectures and communication protocols, specifically Modbus for the app-based UI and REST API for the web-based UI. These test cases were successfully executed on both HMI platforms, yielding outcomes which aligned with the predefined requirements and expected system behaviour, thereby fulfilling the primary objective of this thesis.

This thesis explored various testing approaches, methods, and tools relevant to automated testing. Among the approaches discussed were linear scripting, model-based testing, data-driven testing, keyword-driven testing, Behavior-driven development, hybrid testing and Page object model, each having their own benefits. After evaluating these options, the keyword driven testing approach was chosen for this thesis due to its simplicity and ease of implementation. To design the test suites, three primary methods were employed: error seeding, input partition, and equivalence classes. Additionally, boundary value analysis method was proposed as a future enhancement to improve test coverage of the test suite. For the functional testing scope defined in this thesis, the Robot Framework tool was chosen due to its numerous advantages, which are discussed in the earlier sections. These efforts successfully fulfilled the second objective of this thesis, which was to implement and evaluate testing frameworks, tools, and methods suitable for the automated HMI testing.

To address the third objective outlined in chapter 1, this thesis identifies the challenges related to the reusability and scalability of automated HMI tests. One of the primary solutions proposed is the development of high-level abstracted test cases that are not dependent on any specific system architecture or communication protocols. By decoupling the test logic from hardware and software dependencies, these abstracted test cases can be easily adapted and reused across different HMI platforms within Eaton UPS products. Additionally, by avoiding hardcoded values and designing test cases in a readable format for non-programmers, the test suite becomes more flexible, maintainable, and accessible to a broader range of users.

Lastly, to fulfil the fourth objective of this thesis, a thorough literature review was conducted to explore emerging and promising trends in automation testing, with a particular focus on AI/ML-based testing approaches. This review highlights both the advantages and limitations for these trends. While AI/ML techniques offer significant potential for improving efficiency, adaptability, and defect detection, they also present obstacles such as data quality issues, integration issues, data security and privacy, and algorithmic bias. To address these challenges, the thesis also discusses several practical solutions, including the adoption of data management practices, the use of diverse data sets and data encryptions, and simplifying and optimizing AI-powered tools.

Future advancement of this thesis topic can include the integration of AI, as the involvement of AI and ML is getting common across various industries. Using AI-powered tools like Copilot, human efforts can be reduced for writing the new test cases based on specification document for new features and pre-existing test cases. AI can pick up the trend and logics from pre-existing implementation to prepare the test cases for new features and write a new set of test cases. Additionally, AI can be used in the maintenance of these test cases or related documents. For instance, if a screen or object is removed or added, AI can reflect these changes in the test cases and related documents without any hassle.

References

1. Carrera-Rivera, A., Larrinaga, F., Lasa, G., Reguera-Bakhache, D., & Unamuno, G. (2024). From past to present: Human–machine interfaces evolve toward adaptivity. In Future Perspectives on Human-Computer Interaction Research (pp. 151–186). Springer. https://doi.org/10.1007/978-3-031-71697-3_7
2. Racine, M. S., Parham, J. D., & Rashid, M. H. (2005). An overview of uninterruptible power supplies. In Proceedings of the 37th Annual North American Power Symposium (pp. 159–164). IEEE. https://doi.org/10.1109/NAPS.2005.1560518
3. Automation Community. (2023, December 30). What is a human machine interface (HMI)? Types, advantages. Automation Community. Retrieved March 24, 2025, from https://automationcommunity.com/human-machine-interface/
4. Rajagopal, N. (2013). Automation in display testing – A breakthrough approach (SAE Technical Paper 2013-01-0061). SAE International. https://doi.org/10.4271/2013-01-0061
5. Murash, S. (2025, February 19). Mastering User Interface Testing: Automation, Tools and Tips. Eleken. Retrieved May 12, 2025, from https://www.eleken.co/blog-posts/user-interface-testing
6. Deshmukh, P. S., Date, S. S., Mahalle, P. N., & Barot, J. (2023). Automated GUI Testing for Enhancing User Experience (UX): A Survey of the State of the Art. In ICT Systems and Sustainability (pp. 619-628). Springer. https://link.springer.com/chapter/10.1007/978-981-99-5652-4_55
7. Alégroth, E. (2015). Visual GUI testing: Automating high-level software testing in industrial practice [Doctoral thesis, Chalmers University of Technology and Göteborg University]. Chalmers University of Technology. Retrieved March 28, 2025, from https://publications.lib.chalmers.se/records/fulltext/221145/221145.pdf
8. King, T. M., Ganti, A. S., & Froslie, D. (2011). Enabling automated integration testing of cloud application services in virtualized environments. In Proceedings of the 2011 Conference of the Center for Advanced Studies on Collaborative Research (pp. 120–132). IBM Corp. https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=3831e8d60cc667a61703adc616d196c0747bd869
9. Berner, S., Weber, R., & Keller, R. K. (2005). Observations and lessons learned from automated testing. In Proceedings of the 27th International Conference on Software Engineering (pp. 571–579). ACM. https://doi.org/10.1145/1062455.1062556
10. Sommerville, I. (2004). Software engineering. International computer science series.

https://engineering.futureuniversity.com/BOOKS%20FOR%20IT/Software-Engineering-9th-Edition-by-Ian-Sommerville.pdf
11. Lowell, C., & Stell-Smith, J. (2003). Successful automation of GUI driven acceptance testing. In Proceedings of the 4th International Conference on Extreme Programming and Agile Processes in Software Engineering (XP 03) (pp. 331–333). Berlin, Heidelberg. https://dl.acm.org/doi/10.5555/1763875.1763928
12. Cheung, H. Y. K. (2019). Hardware-in-the-loop test automation setup for power converter system development [Master's thesis, LUT School of Energy Systems, Electrical Engineering] https://lutpub.lut.fi/handle/10024/159150
13. Eaton. (2018). Power infrastructure solutions and products catalogue 2018. Eaton. Retrieved May 13, 2025, from https://www.eaton.com/content/dam/eaton/products/
14. Bekiarov, S. B., & Emadi, A. (2002). Uninterruptible power supplies: Classification, operation, dynamics, and control. In Seventeenth Annual IEEE Applied Power Electronics Conference and Exposition (APEC) (Cat. No.02CH37335) (pp. 10-14). Dallas, TX, USA: IEEE. https://doi.org/10.1109/APEC.2002.989305
15. Joseph, T. (2024, November 11). What is user interface (UI) testing and its scope in 2025. Medium. Retrieved May 14, 2025, from https://medium.com/@timothyjosephcw/what-is-user-interface-ui-testing-and-its-scope-in-2025-437e94d57045
16. Eaton. (2018). Eaton 93PM UPS 30–250 kVA: User's and installation guide (Rev. 006, Document ID: P-164000249). Eaton. Retrieved June 5, 2025, from https://www.eaton.com/content/dam/eaton/products/backup-power-ups-surge-it-power-distribution/backup-power-ups/eaton-93pm-ups/eaton-93pm-ups-30-250kva-users-and-installation-guide-p164000249-en.pdf
17. Abdul Halim, N. S., Abdul Rahman, N. F., Baharom, R., & Mohd Zainuri, M. A. A. (2021). Single-phase transformerless line interactive uninterruptable power supply with two independent control algorithms. Journal of Physics: Conference Series, 1874(1), 012002. https://doi.org/10.1088/1742-6596/1874/1/012002
18. Aamir, M., Kalwar, K. A., & Mekhilef, S. (2016). Review: Uninterruptible power supply (UPS) system. Renewable and Sustainable Energy Reviews, 58, 1395–1410. https://doi.org/10.1016/j.rser.2015.12.335
19. Kumari, B., Chauhan, N., & Vedpal. (2018). A comparison between manual testing and automated testing. Journal of Emerging Technologies and Innovative Research (JETIR), 5(12), 323–327. Retrieved June 5, 2025, from https://www.jetir.org/papers/JETIR1812949.pdf

20. Graham, D., & Fewster, M. (2012). Experiences of test automation: Case studies of software test automation. Addison-Wesley Professional.
21. Khankhoje, R. (2023). An In-Depth Review of Test Automation Frameworks: Types and Trade-offs. International Journal of Advanced Research in Science, Communication and Technology. http://dx.doi.org/10.48175/IJARSCT-13108
22. Mariani, L., Pezzè, M., Riganelli, O., & Santoro, M. (2014). Automatic testing of GUI-based applications. Software Testing, Verification & Reliability, 24(4), 341–366. https://doi.org/10.1002/stvr.1538
23. Alégroth, E., Feldt, R., & Ryrholm, L. (2014). Visual GUI testing in practice: Challenges, problems and limitations. Empirical Software Engineering, 20(3), 694–744. https://doi.org/10.1007/s10664-013-9293-5
24. Chinnapongse, V., Lee, I., Sokolsky, O., Wang, S., & Jones, P. L. (2009). Model-based testing of GUI-driven applications. In S. Lee & P. Narasimhan (Eds.), Software technologies for embedded and ubiquitous systems (Vol. 5860, pp. 101–114). Springer. https://doi.org/10.1007/978-3-642-10265-3_19
25. Khankhoje, R. (2024). AI in test automation: Overcoming challenges, embracing imperatives. International Journal on Soft Computing, Artificial Intelligence and Applications (IJSCAI), 13(1), 1–15. https://doi.org/10.5121/ijscai.2024.13101
26. Pandhare, H. V. (2025). Future of software test automation using AI/ML. International Journal of Engineering and Computer Science, 14(5), 27159–27182. https://doi.org/10.18535/ijecs/v14i05.5139
27. Smith, R. (2007). An overview of the Tesseract OCR engine. Google Inc. Retrieved from https://research.google.com/pubs/archive/33418.pdf
28. Zhang Juan, L. Cai, W. Tong, Yuan Song and Li Ying, "Test Case Reusability Metrics Model," 2010 2nd International Conference on Computer Technology and Development, Cairo, 2010, pp. 294-298. https://doi.org/10.1109/ICCTD.2010.5645869.
29. Merz, Florian & Sinz, Carsten & Post, Hendrik & Gorges, Thomas & Kropf, Thomas. (2015). Bridging the gap between test cases and requirements by abstract testing. Innovations in Systems and Software Engineering. 11. https://link.springer.com/article/10.1007/s11334-015-0245-7
30. Robot Framework Foundation. (2025). Robot Framework User Guide (Version 7.3.2). Retrieved from https://robotframework.org/robotframework/latest/RobotFrameworkUserGuide.html

31. SmartBear Software. (2025, August 13). Zephyr | TestComplete Documentation. SmartBear. https://support.smartbear.com/testcomplete/docs/other-tools/zephyr.html
32. Dhiman, S., & Sharma, P. (2016). Performance testing: A comparative study and analysis of web service testing tools. International Journal of Computer Science and Mobile Computing, 5(6), 507–512. https://www.ijcsmc.com
33. Forgács, I., & Kovács, A. (2023). Modern software testing techniques: A practical guide for developers and testers. Apress. https://doi.org/10.1007/978-1-4842-9893-0